

BUFFER (Local Variables)

Milenko Burlica

28 July, 2026

Table of Contents

1	Core Engine: BUFFER (Local Variables)	3
1.1	Globales Feature: condition (Bedingte Ausführung)	3
1.2	Unterstützte Operationen (op)	3
1.2.1	1. Operation: EXEC (Variablen-Deklaration & Wert-Zuweisung).....	3
1.2.2	2. Operation: ASSERT (Validierung)	4
1.3	💡 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling	5
1.3.1	1. Warum müssen geschweifte Klammern { } in Anführungszeichen?	5
1.3.2	2. Das Problem mit dem Doppelpunkt :	5
1.3.3	3. Maskierung (Escaping) von Anführungszeichen	5
1.3.4	4. Mehrzeilige Texte (Die Block-Notation)	5

1 Core Engine: BUFFER (Local Variables)

Die BUFFER-Engine dient zur Deklaration, Verwaltung und Validierung von lokalen Variablen (Laufzeit-Buffer) innerhalb eines TestFalls über einfache YAML-Konfigurationen.

Ein zentrales Merkmal ist die strikte Entkopplung von Zuweisung und Validierung: Ein Wert wird einmalig im Speicher abgelegt (`op: EXEC`) und unter einem frei wählbaren Variablen-Namen (`name`) registriert. Anschließend können beliebig viele Prüfungen (`op: ASSERT`) auf diesem gespeicherten Wert durchgeführt werden, ohne den Wert neu berechnen oder die ursprüngliche Quelle erneut abfragen zu müssen.

1.1 Globales Feature: condition (Bedingte Ausführung)

Mit dem optionalen Feld `condition` kann gesteuert werden, ob ein Testschritt überhaupt ausgeführt werden soll. Wenn die Bedingung nicht zutrifft (`false`), wird der Schritt einfach übersprungen (`SKIPPED`) und der Test läuft ohne Fehler weiter.

- **Dynamische Werte:** In der Bedingung können alle Platzhalter (wie `{B[...]}` für TestCase-Variablen oder `{E[...]}` für Umgebungsvariablen) verwendet werden.
- **Wichtig für Strings:** Da Textwerte direkt eingesetzt werden, müssen Text-Variablen in einfache Anführungszeichen `'...'` gesetzt werden.

YAML

```
# Wird nur ausgeführt, wenn die globale Umgebungsvariable 'STAGE' auf 'INT' steht
- type: BUFFER
  condition: "'{E[STAGE]}' == 'INT'"
  op: EXEC
  name: "routing_suffix"
  value: "-internal"
```

1.2 Unterstützte Operationen (op)

1.2.1 1. Operation: EXEC (Variablen-Deklaration & Wert-Zuweisung)

Nimmt einen rohen Text-Wert oder einen dynamisch zusammengesetzten String entgegen, löst alle enthaltenen Platzhalter auf und speichert das finale Ergebnis im TestCase-Kontext ab. Bestehende Variablen mit demselben Namen werden dabei überschrieben.

- **Variablen-Auflösung:** Im Feld `value` können alle Dynamic-Tags des Frameworks genutzt werden (`{B[...]}` für TestCase-Variablen oder `{R[...]}` / `{T[...]}` für Parameter und Tabellen-Werte).

YAML

```
- type: BUFFER
  op: EXEC
  name: "full_system_string"
  value: "{B[local_prefix]}-{B[app_name]}-v2"
```

1.2.2 2. Operation: ASSERT (Validierung)

Prüft den Inhalt einer zuvor gespeicherten Variable gegen ein definiertes Kriterium. Schlägt eine Prüfung fehl, bricht der gesamte Test sofort ab (**Fail-Fast**).

Aktion (action)	Beschreibung	Verwendung im YAML
EQUALS	Prüft auf exakte String-Gleichheit.	expected: "ACTIVE"
NOT_EQUALS	Prüft, ob der Wert ungleich dem erwarteten String ist.	expected: "TERMINATED"
CONTAINS	Prüft, ob der erwartete Teilstring im Buffer existiert.	expected: "Payment"
IS_NULL	Prüft, ob die Variable <code>null</code> ist.	kein expected Feld
IS_NOT_NULL	Prüft, ob die Variable existiert (nicht <code>null</code> ist).	kein expected Feld
IS_EMPTY	Prüft, ob der String leer ("") oder <code>null</code> ist.	kein expected Feld
IS_NOT_EMPTY	Prüft, ob der String Text-Inhalt besitzt.	kein expected Feld

YAML

```
# Beispiel: Inhaltsprüfung auf der zuvor gespeicherten Variable
"full_system_string"
- type: BUFFER
  op: ASSERT
  name: "full_system_string"
  action: EQUALS
  expected: "PRE-PROD-Payment-Processing-v2"
```

1.3 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling

Das Schreiben von dynamischen Werten in YAML erfordert Aufmerksamkeit, da bestimmte Zeichen eine strukturelle Bedeutung für den YAML-Parser besitzen.

1.3.1 1. Warum müssen geschweifte Klammern { } in Anführungszeichen?

In YAML leiten geschweifte Klammern am Zeilenanfang ein Inline-Objekt (eine Map) ein. Wenn du einen Platzhalter wie `{B[my_var]}` als Wert einträgst, wirft der YAML-Parser einen Syntaxfehler.

- **Falsch:** `value: {B[local_prefix]}-v2` (Parser stürzt ab)
- **Richtig:** `value: "{B[local_prefix]}-v2"` (Als String geschützt)

1.3.2 2. Das Problem mit dem Doppelpunkt :

Der Doppelpunkt ist das Trennzeichen zwischen Schlüssel und Wert in YAML. Wenn deine Zuweisung einen Doppelpunkt enthält (z. B. zusammengesetzte Protokoll-IDs oder strukturierte Text-Muster), muss der gesamte Wert in Anführungszeichen gesetzt werden.

- **Falsch:** `value: transaction:status:approved`
- **Richtig:** `value: "transaction:status:approved"`

1.3.3 3. Maskierung (Escaping) von Anführungszeichen

Wenn der zuzuweisende Wert selbst doppelte Anführungszeichen enthalten soll (z. B. ein harter JSON-String), gibt es zwei saubere Optionen:

- **Option A (Einfache äußere Anführungszeichen):** Umschließe den gesamten String mit einfachen Anführungszeichen (`'`), dann kannst du innen frei mit doppelten Anführungszeichen arbeiten.

YAML

```
value: '{"status": "ACTIVE", "code": 200}'
```

- **Option B (Backslash-Escaping):** Wenn du doppelte Anführungszeichen außen nutzt, musst du die inneren Anführungszeichen mit `\` maskieren.

YAML

```
value: "{\"status\": \"ACTIVE\", \"code\": 200}"
```

1.3.4 4. Mehrzeilige Texte (Die Block-Notation |)

Wenn du einen längeren strukturierten Textblock, SQL-Queries oder mehrzeilige Payloads in einer Variable zwischenspeichern willst, nutze das Pipe-Zeichen (`|`). YAML liest alles darunter eingerückt als zusammenhängenden Textblock ein, ohne dass Anführungszeichen maskiert werden müssen.

YAML

```
- type: BUFFER
  op: EXEC
  name: "json_payload_template"
  value: |
    {
      "environment": "{B[local_prefix]}",
      "service": "{B[app_name]}",
      "secured": true
    }
```